

Enumeración razonada para escalada Linux

Objetivos de aprendizaje

Convertir salidas de enumeración en relaciones verificables de permisos, priorizar por precondiciones y descartar falsos positivos antes de ejecutar una técnica.

Prerrequisitos

UID real/efectivo, grupos, permisos, procesos, filesystem, entorno y alcance de laboratorio.

Fundamentos técnicos

Un hallazgo no es un vector hasta completar esta cadena:

objeto/proceso privilegiado -> operación que realiza -> dato que controlo
-> momento/condición de consumo -> identidad efectiva -> capacidad obtenida

El color de una herramienta indica prioridad heurística, no explotación ni impacto.

Modelo mental

Para cada pista anota consumidor privilegiado, superficie controlable, frontera rota, precondiciones, prueba inocua y control negativo.

Superficie de ataque

Reglas sudo, ejecutables SUID/capabilities, tareas, servicios, grupos, credenciales, ficheros sensibles, rutas/bibliotecas y kernel. El orden depende de evidencia y coste, no de una lista rígida.

Cómo identificarla

- Un proceso de UID superior consume un archivo, nombre, ruta u opción controlable.
- La precondición se demuestra con permisos efectivos, no solo ls aislado.
- Un marcador inocuo aparece con identidad esperada.
- El control sin influencia no produce el efecto.

Preguntas que debo hacerme

1. ¿Qué identidad obtendría realmente?
2. ¿Qué controlo: contenido, nombre, directorio, argumento o entorno?
3. ¿Quién lo consume y cuándo?
4. ¿Qué defensa anula la relación?
5. ¿Cuál es la capacidad mínima demostrable?

Prueba mínima

id, grupos, sudo -l, montajes y procesos aportan contexto. Para una pista concreta, usa un marcador reversible que pruebe consumo e identidad sin crear shell ni modificar datos sensibles.

Construcción progresiva del payload

1. Registrar identidad actual.
2. Formular relación concreta.
3. Verificar cada precondición por separado.
4. Observar consumidor privilegiado.
5. Introducir marcador inocuo.

6. Confirmar identidad y limpiar.

Anatomía de los payloads

- Contexto de entrada: pspy muestra `/usr/local/bin/backup` con UID 0.
- Sintaxis original: el script lee `/opt/team/job.conf`.
- Entrada controlada: fichero de configuración escribible por grupo.
- Transformaciones conocidas: parser key/value.
- Parser final: script de backup.
- Sink: creación de archivo como root.
- Primitiva: cambiar solo la ruta de salida a un marcador permitido.
- Payload mínimo: valor `/tmp/backup-proof`.
- Significado de cada componente: `/tmp` satisface la raíz permitida; `backup-proof` identifica la prueba.
- Resultado esperado: el consumidor privilegiado crea el marcador con UID 0.
- Control negativo: configuración sin cambio.
- Restricción observada: solo rutas bajo `/tmp` aceptadas.
- Por qué falla la variante básica: una ruta fuera de `/tmp` es rechazada por el parser.
- Hipótesis de adaptación: medir la capacidad mínima dentro de la raíz admitida.
- Payload adaptado: `/tmp/backup-proof` con token único.
- Por qué debería funcionar: conserva la gramática aceptada y cambia solo el destino observable.
- Evidencia: archivo UID 0 con contenido esperado.
- Cuándo no funcionaría: config releída por otro usuario, firma, permisos o ruta fija.

Variaciones según el contexto

Automatización encuentra amplitud; comprobaciones manuales explican causalidad. Versiones, mounts, contenedores, MAC y `no_new_privs` pueden cambiar la validez de una pista.

Filtros y bypasses

No existe “bypass” general. Si falla una precondition, descarta o busca otra relación. No fuerces una receta porque un nombre aparezca en una base de datos.

Evidencias de confirmación

Consumo controlado por una identidad superior con efecto mínimo atribuible. NOPASSWD, un bit SUID o un directorio escribible son hallazgos, no confirmación de escalada.

Escalado de impacto

Después del marcador, evaluar la capacidad exacta: lectura, escritura, ejecución o cambio de identidad. Ampliar solo si el objetivo del laboratorio lo exige.

Errores frecuentes

- Priorizar colores sin consumidor.
- Ejecutar herramientas agresivas antes de contexto básico.
- Confundir propietario con UID efectivo del consumidor.
- Ignorar mount, entorno y versión.
- Saltar al payload final.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
fichero escribible, nunca leído	falso positivo	observar accesos/mtime/proceso
proceso periódico no es root	impacto lateral, no root	UID real del proceso
SUID no cambia EUID	nosuid, no_new_privs o binario baja privilegio	mount, /proc, id -ru/-u
sudo regla no coincide	argumentos/ruta/host	leer salida completa de sudo -l
herramienta marca kernel	versión parcheada/backport	paquete y advisory del proveedor

Mitigaciones

Privilegio mínimo, permisos y propietarios correctos, rutas absolutas, entornos controlados, revisión de tareas/reglas y monitorización de cambios.

Relación con pentesting y certificaciones

Se evalúa priorización y prueba causal, no cantidad de comandos ejecutados.

Caso guiado

Caso A — Básico: configuración consumida por root

pspy muestra un backup root cada minuto; su config es escribible por el grupo del alumno. Observación: consumidor y control coinciden. Experimento: cambiar ruta de salida a marcador y esperar un ciclo. Resultado: archivo root. Conclusión: primitiva de escritura privilegiada confirmada. Siguiendo paso: evaluar impacto mínimo.

Caso de adaptación

Caso B — Hallazgo rojo sin consumidor

linpeas marca /opt/team escribible, pero ninguna tarea, servicio ni binario lo referencia. Experimento: búsqueda de referencias y observación de procesos. Resultado: cero consumo. Conclusión: no es vector actual; no se inventa payload.

Caso C — Transferencia: socket de mantenimiento

Un grupo puede hablar con un socket de daemon root. El alumno debe identificar operaciones permitidas y usar una acción status antes de cualquier cambio. La primitiva es controlar una operación privilegiada, no escribir un fichero.

Caso D — Falso positivo: SUID del propio usuario

Un binario tiene SUID, pero propietario y EUID son el mismo usuario actual. Experimento: comparar UID real/efectivo. Conclusión: no hay ganancia de identidad.

Ejercicios

1. Prioriza cinco hallazgos por consumidor, control y capacidad.
2. Diseña un marcador para una tarea periódica.
3. Explica qué evidencia descarta un directorio escribible.
4. Separa hallazgo, primitiva e impacto en una regla sudo.

Resumen

Enumerar es construir relaciones verificables. La herramienta propone pistas; el alumno demuestra precondiciones, identidad y capacidad.

Chuleta operativa

1. Identidad.
2. Consumidor privilegiado.
3. Superficie controlada.
4. Momento de consumo.
5. Marcador y control.
6. Capacidad exacta.

Referencias

- [Linux credentials\(7\)](#)
- [Concepto interno](#) (enum-privesc-linux)

Navegación

Índice: [curso](#). Siguiendo: [Abuso de sudo](#). Práctica: [cuaderno](#).